

Assignment Details:

Practical 1 to 5 (Dhruval Gamit)

Assignment - 1

Practical - 1 : Install Node.js (LTS) and verify the installation by running node -v and npm -v in your terminal. Take a screenshot of the output.

Step 1: Download Node.js (LTS)

1. Open your web browser.
2. Go to the official Node.js website:
 - <https://nodejs.org>
3. You'll see two versions:
 - **LTS (Long-Term Support)**
 - Current
4. Click **Download Node.js (LTS)**.

Step 2: Install Node.js

1. Open the downloaded .msi installer.
2. Click **Next**.
3. Accept the License Agreement.
4. Click **Next**.
5. Leave the default installation location unchanged.
6. Click **Next**.
7. Keep all default features selected.
8. Click **Next**.
9. Leave the option for **Automatically install the necessary tools** unchecked unless instructed by your course.
10. Click **Install**.
11. If Windows asks for permission, click **Yes**.
12. Wait until installation finishes.
13. Click **Finish**.

Step 3: Open Terminal and Verify Node.js Installation

Type

node -v

Press Enter.

Output

```
PS C:\Users\VICTUS> node -v
v24.15.0
PS C:\Users\VICTUS>
```

Step 4: Verify npm Installation

Type

npm -v

Press Enter.

Output

```
PS C:\Users\VICTUS> npm -v
11.12.1
PS C:\Users\VICTUS> |
```

If Commands Don't Work

If you get

'node' is not recognized...

Restart your computer first.

If it still doesn't work:

1. Uninstall Node.js.
2. Install it again.
3. Ensure **Add to PATH** is enabled (it usually is by default).
4. Open a new Command Prompt and try again.

Practical - 2 : Install Angular CLI globally using npm and verify it with ng version. Note down the Angular CLI version installed on your machine.

Prerequisites

Before starting, make sure:

- Node.js is installed.
- npm is working.

Verify by running:

```
node -v  
npm -v
```

If both commands display version numbers, continue.

Step 1: Open Command Prompt

Press **Windows + R**, type:

```
cmd
```

and press **Enter**.

Step 2: Install Angular CLI Globally

Type the following command:

```
npm install -g @angular/cli
```

Press **Enter**.

What this command does

- npm → Node Package Manager
- install → Installs a package
- -g → Installs it globally so you can use ng from any folder
- @angular/cli → The Angular Command Line Interface package

Step 3: Wait for Installation

The installation may take **2–5 minutes**, depending on your internet speed.

You may see output similar to:

added 250 packages in 40s

or

changed 250 packages

This indicates the installation completed successfully.

Step 4: Verify Angular CLI Installation

Run:

`ng version`

or

`ng v`

output:

```
PS C:\Users\VICTUS> ng version

Angular CLI
Node.js
Package Manager
Operating System

: 22.0.8
: 24.15.0
: npm 11.12.1
: win32 x64
```

Both commands display the installed Angular CLI version.

If You Get an Error

Error:

'ng' is not recognized as an internal or external command

Try these steps:

1. Close the Command Prompt.
2. Open a new Command Prompt and run: **ng version**
3. If it still doesn't work, restart your computer.
4. If the issue persists, check whether Angular CLI is installed:
npm list -g @angular/cli
5. If it's not installed, run the installation command again:
npm install -g @angular/cli

Practical - 3 : Create a new Angular project called my-first-app, run it using ng serve – open, and confirm it loads at http://localhost:4200.

Step 1: Open Command Prompt

Press **Windows + R**, type:

cmd

Press **Enter**.

Step 2: Go to the Folder Where You Want to Create the Project

For example, if you want to create it in the **Documents** folder:

```
cd %USERPROFILE%\Documents
```

Or create it on the D: drive:

D:

```
cd Projects
```

Choose any location where you want to save the project.

Step 3: Create a New Angular Project

Run:

```
ng new my-first-app
```

Press **Enter**.

Step 4: Answer the Setup Questions

Depending on your Angular CLI version, you may be asked a few questions.

Example Questions

1. Would you like to create a zoneless application?

Select:

No

(or press Enter if No is the default)

2. Which stylesheet format would you like to use?

Use the **Arrow Keys** to select:

CSS

Then press **Enter**.

If your CLI asks slightly different questions, the default options are usually fine unless your instructor specified otherwise.

Step 5: Wait for the Project to Be Created

Angular will:

- Create the project folder.
- Install required packages.
- Display a success message.

You should see something similar to:

- ✓ Packages installed successfully.
- ✓ Project 'my-first-app' created successfully.

This may take a few minutes.

```
PS D:\Projects> ng new my-first-app
✓Which stylesheet system would you like to use? CSS [ https://developer.mozilla.org/docs/Web/CSS ]
✓Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? No
✓Which AI tools do you want to configure with Angular best practices? https://angular.dev/ai/develop-with-ai None
CREATE my-first-app/.prettierrc (173 bytes)
CREATE my-first-app/angular.json (1994 bytes)
CREATE my-first-app/package.json (788 bytes)
CREATE my-first-app/README.md (1522 bytes)
CREATE my-first-app/tsconfig.json (939 bytes)
CREATE my-first-app/.editorconfig (331 bytes)
CREATE my-first-app/.gitignore (666 bytes)
CREATE my-first-app/tsconfig.app.json (412 bytes)
CREATE my-first-app/tsconfig.spec.json (456 bytes)
CREATE my-first-app/.vscode/extensions.json (134 bytes)
CREATE my-first-app/.vscode/launch.json (490 bytes)
CREATE my-first-app/.vscode/tasks.json (1020 bytes)
CREATE my-first-app/src/main.ts (228 bytes)
CREATE my-first-app/src/index.html (309 bytes)
CREATE my-first-app/src/styles.css (81 bytes)
CREATE my-first-app/src/app/app.spec.ts (702 bytes)
CREATE my-first-app/src/app/app.ts (306 bytes)
CREATE my-first-app/src/app/app.css (0 bytes)
CREATE my-first-app/src/app/app.html (20488 bytes)
CREATE my-first-app/src/app/app.config.ts (324 bytes)
CREATE my-first-app/src/app/app.routes.ts (80 bytes)
CREATE my-first-app/public/favicon.ico (15086 bytes)
✓Packages installed successfully.
warning: in the working copy of 'package-lock.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git touches it
Successfully initialized git.
PS D:\Projects>
```


Step 6: Move into the Project Folder

Run:

```
cd my-first-app
```

Step 7: Start the Development Server

Run:

```
ng serve --open
```

or the shorter form:

```
ng serve -o
```

```
PS D:\Projects> cd my-first-app
PS D:\Projects\my-first-app> ng serve --open

Would you like to share pseudonymous usage data about this project with the Angular Team
at Google under Google's Privacy Policy at https://policies.google.com/privacy. For more
details and how to change this setting, see https://angular.dev/cli/analytics.

No
Global setting: enabled
Local setting: disabled
Effective status: disabled
Initial chunk files | Names | Raw size
main.js | main | 48.48 kB |
styles.css | styles | 95 bytes |
| Initial total | 48.57 kB

Application bundle generation complete. [4.424 seconds] - 2026-07-26T06:01:13.335Z

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
→ Local: http://localhost:4200/
→ press h + enter to show help
```

Step 8: Wait for Compilation

You should see output similar to:

✓ Compiled successfully.

and

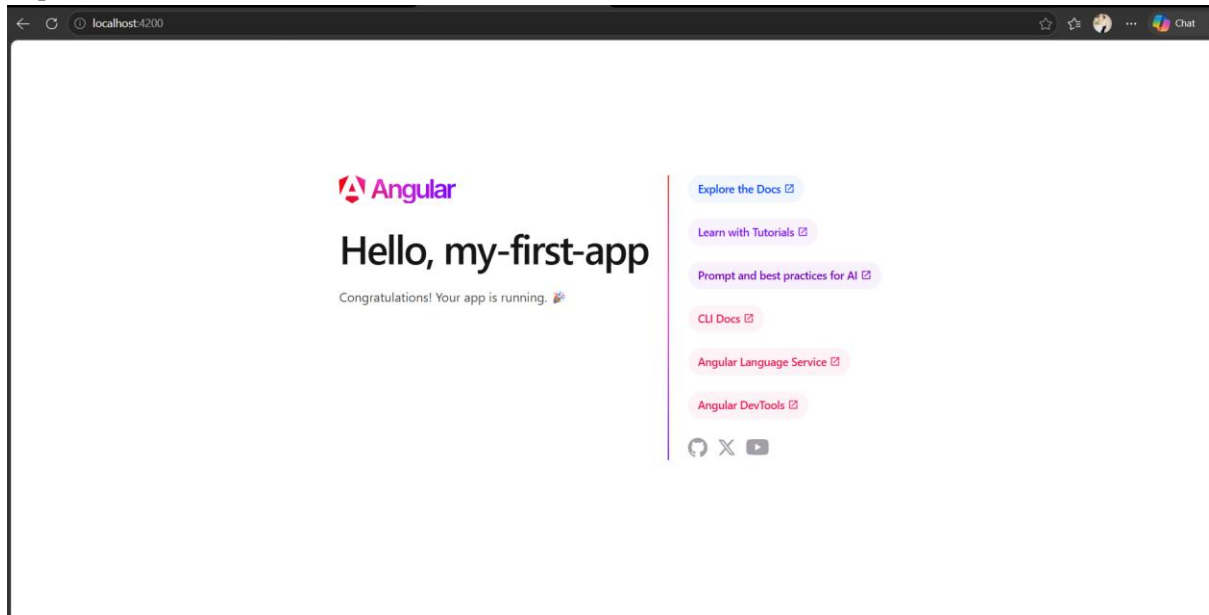
Local: http://localhost:4200/

Keep this Command Prompt window open while the app is running.

Step 9: Browser Opens Automatically

Your browser should open to:

`http://localhost:4200`



You should see the default Angular welcome page (the exact appearance depends on the Angular version).

If the Browser Doesn't Open Automatically

Open your browser manually and visit:

`http://localhost:4200`

If Port 4200 Is Already in Use

You may see:

Port 4200 is already in use.

Press:

Y

Angular will use another port (for example, `http://localhost:4201`).

Assignment - 2

Practical - 1 : To create a TypeScript file named `datatypes.ts` and demonstrate the use of different TypeScript data types such as **string, number, boolean, array, tuple, any, and unknown**, and display their values using `console.log()`.

Software Requirements

- Operating System: Windows 10/11
- Node.js (LTS)
- Angular CLI
- Visual Studio Code
- TypeScript Compiler (tsc)

Procedure

1. Open the existing Angular project (**my-first-app**) in Visual Studio Code.
2. Create a new file named **datatypes.ts** inside the **src** folder.
3. Write TypeScript code demonstrating the following data types:
 - String
 - Number
 - Boolean
 - Array
 - Tuple
 - Any
 - Unknown
4. Save the file.
5. Open the integrated terminal in Visual Studio Code.
6. Navigate to the **src** folder if necessary.

Compile the TypeScript file using the following command:

`npx tsc datatypes.ts --ignoreConfig`

7. *(The `--ignoreConfig` option is used because newer versions of TypeScript show an error when a file is compiled directly while a **`tsconfig.json`** file exists in the Angular project.)*
8. After successful compilation, run the generated JavaScript file using:
`node datatypes.js`
9. Observe the output displayed in the terminal.
10. Capture a screenshot showing the source code and terminal output.

Source Code

```
// String
let studentName: string = "Dhruval";
```

```
// Number
```

```
let age: number = 21;

// Boolean
let isStudent: boolean = true;

// Array
let subjects: string[] = ["Angular", "TypeScript", "HTML", "CSS"];

// Tuple
let studentInfo: [number, string] = [101, "Dhruval"];

// Any
let data: any = "Hello";
data = 500;
data = false;

// Unknown
let value: unknown = "Angular";

console.log("String:", studentName);
console.log("Number:", age);
console.log("Boolean:", isStudent);
console.log("Array:", subjects);
console.log("Tuple:", studentInfo);
console.log("Any:", data);
console.log("Unknown:", value);
```

Commands Used

```
npx tsc datatypes.ts --ignoreConfig
node datatypes.js
```

Expected Output

```
PS D:\Projects\my-first-app\src> npx tsc datatypes.ts --ignoreConfig
PS D:\Projects\my-first-app\src> node datatypes.js
String: Dhruval
Number: 21
Boolean: true
Array: [ 'Angular', 'TypeScript', 'HTML', 'CSS' ]
Tuple: [ 101, 'Dhruval' ]
Any: false
Unknown: Angular
PS D:\Projects\my-first-app\src> |
```

Result

The `datatypes.ts` file was created successfully. Different TypeScript data types—**string**, **number**, **boolean**, **array**, **tuple**, **any**, and **unknown**—were declared and initialized. The file was compiled successfully using the TypeScript compiler, and all values were displayed correctly in the terminal using `console.log()`. Thus, the objective of the assignment was achieved successfully.

Practical - 2 : Create an Interface named IStudent containing studentId, studentName, course, and optional email. Create two student objects and display their details.

Step 1: Open Your Angular Project

Open **Visual Studio Code**.

Open your Angular project:

my-first-app

Step 2: Create a New TypeScript File

Inside the **src** folder, create a new file named:

student.ts

Step 3: Write the Following Code

// Interface Definition

```
interface IStudent {  
  
    studentId: number;  
  
    studentName: string;  
  
    course: string;  
  
    email?: string; // Optional property  
  
}
```

// Student Object 1

```
let student1: IStudent = {  
  
    studentId: 101,  
  
    studentName: "Dhruval Gamit",  
  
    course: "Angular",  
  
    email: "dhruval@example.com"  
  
};
```

```
// Student Object 2
```

```
let student2: IStudent = {  
  
    studentId: 102,  
  
    studentName: "Rahul Patel",  
  
    course: "TypeScript"  
  
};
```

```
// Display Student 1 Details
```

```
console.log("Student 1 Details");  
  
console.log("Student ID:", student1.studentId);  
  
console.log("Student Name:", student1.studentName);  
  
console.log("Course:", student1.course);  
  
console.log("Email:", student1.email);  
  
  
console.log("-----");
```

```
// Display Student 2 Details
```

```
console.log("Student 2 Details");  
  
console.log("Student ID:", student2.studentId);  
  
console.log("Student Name:", student2.studentName);  
  
console.log("Course:", student2.course);  
  
console.log("Email:", student2.email);
```

Save the file (**Ctrl + S**).

Step 4: Open the Terminal

In VS Code, open:

Terminal → **New Terminal**

or press:

Ctrl + `

Step 5: Go to the src Folder

If you're not already in the `src` folder, run:

```
cd src
```

Step 6: Compile the TypeScript File

Run:

```
npx tsc student.ts --ignoreConfig
```

After successful compilation, a file named **student.js** will be created.

Step 7: Run the Program

Execute:

```
node student.js
```

Step 8: Verify the Output

```
PS D:\Projects\my-first-app\src> npx tsc student.ts --ignoreConfig
PS D:\Projects\my-first-app\src> node student.js
Student 1 Details
Student ID: 101
Student Name: Dhruval Gamit
Course: Angular
Email: dhruval@example.com
-----
Student 2 Details
Student ID: 102
Student Name: Rahul Patel
Course: TypeScript
Email: undefined
PS D:\Projects\my-first-app\src> |
```


Practical - 2 : Create a Class named Employee with id, name, and salary. Add a constructor and a method named displayEmployee(). Create two employee objects and display their information.

Step 1: Open Your Angular Project

Open **Visual Studio Code** and open your Angular project:

my-first-app

Step 2: Create a New TypeScript File

Inside the **src** folder, create a new file named:

employee.ts

Step 3: Write the Following Code

```
// Employee Class

class Employee {

    id: number;

    name: string;

    salary: number;


    // Constructor

    constructor(id: number, name: string, salary: number) {

        this.id = id;

        this.name = name;

        this.salary = salary;

    }


    // Method to Display Employee Details

    displayEmployee(): void {

        console.log("Employee ID:", this.id);
```

```
        console.log("Employee Name:", this.name);

        console.log("Salary:", this.salary);

        console.log("-----");
    }
}

// Create Employee Objects

let emp1 = new Employee(101, "Dhruval Gamit", 50000);

let emp2 = new Employee(102, "Rahul Patel", 60000);


// Display Employee Details

console.log("Employee 1 Details");

emp1.displayEmployee();


console.log("Employee 2 Details");

emp2.displayEmployee();
```

Save the file (**Ctrl + S**).

Step 4: Open the Terminal

In Visual Studio Code:

Terminal → New Terminal

or press:

Ctrl + `

Step 5: Go to the src Folder

If you're not already in the `src` folder, run:

```
cd src
```

Step 6: Compile the TypeScript File

Run:

```
npx tsc employee.ts --ignoreConfig
```

This creates a new file named:

```
employee.js
```

Step 7: Run the Program

Execute:

```
node employee.js
```

Step 8: Verify the Output

```
PS D:\Projects\my-first-app\src> npx tsc employee.ts --ignoreConfig
PS D:\Projects\my-first-app\src> node employee.js
Employee 1 Details
Employee ID: 101
Employee Name: Dhruval Gamit
Salary: 50000
-----
Employee 2 Details
Employee ID: 102
Employee Name: Rahul Patel
Salary: 60000
-----
PS D:\Projects\my-first-app\src> |
```

Assignment - 3

Practical - 1: Components & Data Binding.

T1. Create a component called student-card. Add properties name, course, and isActive in the component class.

Step 1: Open the Terminal

Open the terminal in VS Code.

Step 2: Go to the Project Folder

If you're not already there:

```
cd D:\Projects\my-first-app
```

Step 3: Generate the Component

Run:

```
ng generate component student-card
```

or the short command:

```
ng g c student-card
```

Angular will create:

```
src
├── app
│   └── student-card
│       ├── student-card.css
│       ├── student-card.html
│       ├── student-card.spec.ts
│       └── student-card.ts
```

Step 4: Open

student-card.ts

```
import { Component } from '@angular/core';
```

```
@Component({
  selector: 'app-student-card',
```

```
imports: [],
templateUrl: './student-card.html',
styleUrl: './student-card.css'
})
export class StudentCard {

  name: string = "Dhruval Gamit";
  course: string = "Angular";
  isActive: boolean = true;

}
```

T2. Use Interpolation to display name and course in the template.

Open

student-card.html

```
<h2>Student Information</h2>
```

```
<p>Name: {{ name }}</p>
```

```
<p>Course: {{ course }}</p>
```

Run the project:

```
ng serve -o
```

You should see:

Student Information

Name: Dhruval Gamit

Course: Angular

T3. Use Property Binding to disable a button when isActive is false (e.g., [disabled]="!isActive").

In

student-card.html

add:

```
<button [disabled]="!isActive">  
  Submit  
</button>
```

When

```
isActive = true;
```

the button is enabled.

If

```
isActive = false;
```

the button becomes disabled.

T4. Use Event Binding to create a button that toggles `isActive` when clicked (e.g., `(click)="toggleStatus()"`).

Open

student-card.ts

Add this method:

```
toggleStatus() {  
  this.isActive = !this.isActive;  
}
```

Now update

student-card.html

```
<h2>Student Information</h2>
```

```
<p>Name: {{ name }}</p>
```

```
<p>Course: {{ course }}</p>
```

```
<button [disabled]="!isActive">  
  Submit  
</button>
```

```
<br><br>
```

```
<button (click)="toggleStatus()">  
  Toggle Status  
</button>
```

Clicking **Toggle Status** will enable/disable the **Submit** button.

T5. Add an input field bound to name using Two-Way Binding with [(ngModel)]. (Remember to import FormsModule.)

Step 1: Import FormsModule

Open

app.ts

(or app.module.ts if your project is module-based)

If your project is standalone, modify it like this:

```
import { Component } from '@angular/core';  
import { FormsModule } from '@angular/forms';  
import { StudentCard } from './student-card/student-card';
```

```
@Component({  
  selector: 'app-root',  
  imports: [  
    FormsModule,  
    StudentCard  
  ],  
  templateUrl: './app.html',  
  styleUrls: ['./app.css']  
})  
export class App {  
}
```

If you have `app.module.ts` instead, import `FormsModule` there under the `imports` array.

Step 2: Import `FormsModule` in `student-card.ts`

Update the component:

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
```

```
@Component({
  selector: 'app-student-card',
  imports: [FormsModule],
  templateUrl: './student-card.html',
  styleUrls: ['./student-card.css']
})
export class StudentCard {

  name: string = "Dhruval Gamit";
  course: string = "Angular";
  isActive: boolean = true;

  toggleStatus() {
    this.isActive = !this.isActive;
  }

}
```

Step 3: Update HTML

`student-card.html`

```
<h2>Student Information</h2>
```

```
<p>Name: {{ name }}</p>
```

```
<p>Course: {{ course }}</p>
```

```
<input
  type="text"
  [(ngModel)]="name"
  placeholder="Enter Name">
```

```
<br><br>
```

```
<button [disabled]="!isActive">
```



```
    Submit
  </button>

  <br><br>

  <button (click)="toggleStatus()">
    Toggle Status
  </button>
```

Step 4: Display the Component

Open

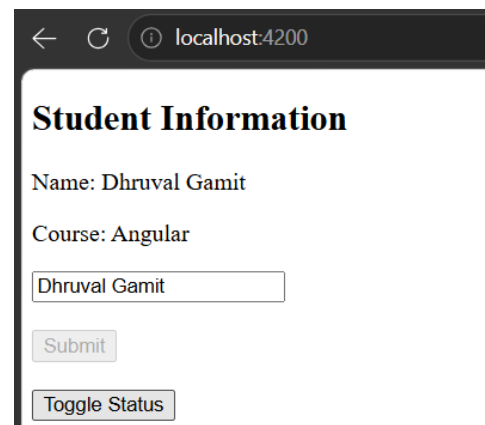
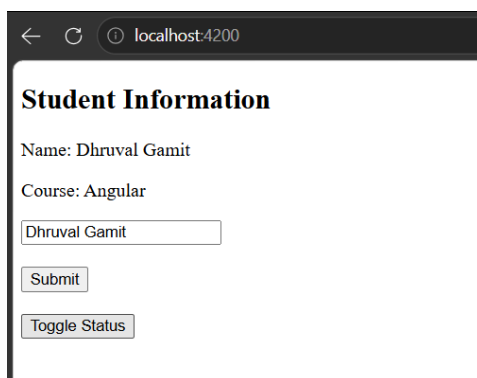
app.html

```
<app-student-card></app-student-card>
```

Step 5: Run the Project

ng serve -o

Open: <http://localhost:4200>



Practical - 2 : Built-in Directives.

D1. Create an array of at least 5 student objects (name, marks). Use *ngFor to display them in a list, showing the index of each item as well.

Step 1: Open

src/app/student-card/student-card.ts

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-student-card',
  imports: [CommonModule],
  templateUrl: './student-card.html',
  styleUrls: ['./student-card.css']
})
export class StudentCard {

  students = [
    { name: 'Dhruval', marks: 85 },
    { name: 'Rahul', marks: 35 },
    { name: 'Priya', marks: 92 },
    { name: 'Amit', marks: 40 },
    { name: 'Neha', marks: 55 }
  ];

}
```

Step 2: Open

student-card.html

Add:

```
<h2>Student List</h2>

<ul>
  <li *ngFor="let student of students; let i = index">
    {{ i + 1 }}.
    {{ student.name }} -
    {{ student.marks }}
  </li>
</ul>
```

Run:

```
ng serve -o
```

Output

Student List

- 1. Dhruval - 85
- 2. Rahul - 35
- 3. Priya - 92
- 4. Amit - 40
- 5. Neha - 55

D2. Using `*ngIf`, show a "Pass" label if a student's marks are greater than 40, otherwise show "Fail". Use the if/else template syntax (`*ngIf="...; else otherBlock"`).

Keep the same `students` array.

```
<h2>Student List</h2>
```

```
<ul>
```

```
<li *ngFor="let student of students; let i = index">
```

```
  {{ i + 1 }}.
```

```
  {{ student.name }}
```

```
-
```

```
  {{ student.marks }}
```

```
<span *ngIf="student.marks > 40; else failBlock">
```

```
  Pass
```

```
</span>
```

```
<ng-template #failBlock>
```

```
  Fail
```

```
</ng-template>
```

```
</li>
```

```
</ul>
```

Output

Student List

- 1. Dhruval - 85 Pass
- 2. Rahul - 35 Fail
- 3. Priya - 92 Pass
- 4. Amit - 40 Fail
- 5. Neha - 55 Pass

D3.Create a component with a status property ("pending", "approved", "rejected"). Use [ngSwitch] on a container element and *ngSwitchCase / *ngSwitchDefault to display a different message for each status.

Step 1

Open

student-card.ts

Add:

```
status = "approved";
```

Now the class becomes:

```
export class StudentCard {  
  
  students = [  
    { name: 'Dhruval', marks: 85 },  
    { name: 'Rahul', marks: 35 },  
    { name: 'Priya', marks: 92 },  
    { name: 'Amit', marks: 40 },  
    { name: 'Neha', marks: 55 }  
  ];  
  
  status = "approved";  
  
}
```

Step 2

At the bottom of student-card.html, add:

```
<hr>
```

```
<h2>Application Status</h2>
```

```
<div [ngSwitch]="status">
```

```
  <p *ngSwitchCase="'pending'">  
    Your application is Pending.  
  </p>
```

```
  <p *ngSwitchCase="'approved'">  
    Your application is Approved.  
  </p>
```

```
  <p *ngSwitchCase="'rejected'">  
    Your application is Rejected.  
  </p>
```

```
  <p *ngSwitchDefault>  
    Unknown Status  
  </p>
```

```
</div>
```

Step 3

Change:

```
status = "approved";
```

to

```
status = "pending";
```

or

```
status = "rejected";
```

Run again to see different outputs.

Final Output

Student List

- 1. Dhruval - 85 Pass
- 2. Rahul - 35 Fail
- 3. Priya - 92 Pass
- 4. Amit - 40 Fail
- 5. Neha - 55 Pass

Application Status

☒ Your application is Approved.

Practical - 3 : Attribute & Custom Directives.

D4.Use Angular's built-in `ngStyle` and `ngClass` attribute directives to dynamically change the background color and add a CSS class to an element based on a condition.

Step 1: Open

`src/app/student-card/student-card.ts`

Add the following properties inside the class:

```
isActive = true;
```

```
export class StudentCard {
```

```
  students = [
```

```
    { name: 'Dhruval', marks: 85 },
```

```
    { name: 'Rahul', marks: 35 },
```

```
    { name: 'Priya', marks: 92 },
```

```
    { name: 'Amit', marks: 40 },
```

```
    { name: 'Neha', marks: 55 }
```

```
  ];
```

```
  status = "approved";
```

```
  isActive = true;
```

```
}
```

Step 2: Open

`student-card.css`

Add:

```
.active {  
    color: white;  
    font-weight: bold;  
    padding: 10px;  
}
```

Step 3: Open

student-card.html

Add:

```
<hr>
```

```
<h2>ngStyle and ngClass Example</h2>
```

```
<div
```

```
    [ngStyle]="{'background-color': isActive ? 'green' : 'red'}"
```

```
    [ngClass]="{'active': isActive}">
```

```
    Student Status
```

```
</div>
```

When `isActive = true`, the background becomes **green** and the CSS class is applied.

ngStyle and ngClass Example



Student Status

If you change it to:

```
isActive = false;
```

the background becomes **red**.

ngStyle and ngClass Example

Student Status

D5.Generate a custom attribute directive called appHighlight using ng generate directive highlight. Make it change an element's background color to yellow on mouseenter and revert it on mouseleave (use @HostListener).

Step 1: Open the Terminal

Go to your project folder:

```
cd D:\Projects\my-first-app
```

Step 2: Generate the Directive

Run:

```
ng generate directive highlight
```

or

```
ng g d highlight
```

Angular creates:

```
src
```

```
└─ app
```

```
    └─ highlight.ts
```

Step 3: Open the Directive File

Replace everything with:

```
import { Directive, ElementRef, HostListener } from '@angular/core';
```

```
@Directive({
```

```
  selector: '[appHighlight]
```

```
})
```

```
export class Highlight {
```

```
  constructor(private element: ElementRef) { }
```

```
  @HostListener('mouseenter')
```

```
    onMouseEnter() {
```

```
      this.element.nativeElement.style.backgroundColor = 'yellow';
```

```
    }
```

```
  @HostListener('mouseleave')
```

```
    onMouseLeave() {
```

```
      this.element.nativeElement.style.backgroundColor = '';
```

```
    }
```

```
}
```

Step 4: Import the Directive

Open

student-card.ts

Import the directive:

```
import { Highlight } from '../highlight';
```

Then update the component decorator:

```
@Component({  
  selector: 'app-student-card',  
  imports: [CommonModule, Highlight],  
  templateUrl: './student-card.html',  
  styleUrls: ['./student-card.css']  
})
```

D6.Apply your appHighlight directive to at least two different elements in a template and confirm the hover effect works.

Open

student-card.html

Add:

```
<hr>
```

```
<h2>Custom Directive Example</h2>
```

```
<h3 appHighlight>
```

```
  Welcome to Angular
```

```
</h3>
```

```
<p appHighlight>
```

Hover over this paragraph to highlight it.

</p>

Step 5: Run the Project

ng serve -o

Result

When you move the mouse over:

Welcome to Angular

its background becomes **yellow**.

Custom Directive Example

Welcome to Angular

Hover over this paragraph to highlight it.

When you move the mouse away, it returns to its original background.

The same behavior occurs for:

Hover over this paragraph to highlight it.

Custom Directive Example

Welcome to Angular

Hover over this paragraph to highlight it.

Practical - 4 : Pipes.

P1. Display today's date in three different formats using the built-in date pipe (e.g., 'short', 'fullDate', 'dd/MM/yyyy').

Step 1: Open

src/app/student-card/student-card.ts

Add the following property inside the class:

```
today = new Date();
```

Step 2: Open

student-card.html

Add:

```
<hr>
```

```
<h2>Date Pipe Example</h2>
```

```
<p>Short Date: {{ today | date:'short' }}</p>
```

```
<p>Full Date: {{ today | date:'fullDate' }}</p>
```

```
<p>Custom Date: {{ today | date:'dd/MM/yyyy' }}</p>
```

Output:

Date Pipe Example

Short Date: 7/31/26, 12:13 PM

Full Date: Friday, July 31, 2026

Custom Date: 31/07/2026

P2.Display a price value using the currency pipe, formatted in USD and INR.

Step 1

Open

student-card.ts

Add:

```
price = 25000;
```

Step 2

Open

student-card.html

Add:

```
<hr>
```

```
<h2>Currency Pipe Example</h2>
```

```
<p>USD: {{ price | currency:'USD' }}</p>
```

```
<p>INR: {{ price | currency:'INR' }}</p>
```

Output:

Currency Pipe Example

USD: \$25,000.00

INR: ₹25,000.00

P3.Take a string property (e.g., a student's name) and display it using the uppercase and lowercase pipes.

Step 1

Open

student-card.ts

Add:

```
studentName = "Dhruval Gamit";
```

Step 2

Open

student-card.html

Add:

```
<hr>
```

```
<h2>Uppercase & Lowercase Pipe</h2>
```

```
<p>Original: {{ studentName }}</p>
```

```
<p>Uppercase: {{ studentName | uppercase }}</p>
```

```
<p>Lowercase: {{ studentName | lowercase }}</p>
```

Output:

Uppercase & Lowercase Pipe

Original: Dhruval Gamit

Uppercase: DHRUVAL GAMIT

Lowercase: dhruval gamit

P4.Create a custom pipe called truncateText that shortens a long string to a given number of characters and appends "...". Use ng generate pipe truncateText, implement the transform() method, and use it in a template.

Step 1: Generate the Pipe

Open the terminal.

Go to your project folder:

```
cd D:\Projects\my-first-app
```

Run:

```
ng generate pipe truncateText
```

or

```
ng g p truncateText
```

Angular creates a file similar to:

```
src
├── app
│   └── truncate-text.pipe.ts
```

Step 2: Open

truncate-text.pipe.ts

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'truncateText'
})
export class TruncateTextPipe implements PipeTransform {

  transform(value: string, limit: number): string {

    if (!value) {
      return "";
    }

    if (value.length <= limit) {
```



```
    return value;
  }

  return value.substring(0, limit) + '...';
}
}
```

Step 3: Import the Pipe

Open

student-card.ts

Import the pipe:

```
import { TruncateTextPipe } from '../truncate-text.pipe';
```

Then update the `imports` array:

```
@Component({
  selector: 'app-student-card',
  imports: [
    CommonModule,
    Highlight,
    TruncateTextPipe
  ],
  templateUrl: './student-card.html',
  styleUrls: ['./student-card.css']
})
```

Step 4

Open

student-card.ts

Add:

```
description = "Angular is a powerful front-end framework developed by Google for building modern web applications.";
```

Step 5

Open

student-card.html

Add:

```
<hr>
```

```
<h2>Custom Pipe Example</h2>
```

```
<p>Original:</p>
```

```
<p>{{ description }}</p>
```

```
<p>Truncated:</p>
```

```
<p>{{ description | truncateText:30 }}</p>
```

Step 6: Run the Project

```
ng serve -o
```

Output:

Custom Pipe Example

Original:

Angular is a powerful front-end framework developed by Google for building modern web applications.

Truncated:

Angular is a powerful front-en...

Assignment - 4

Practical - 1 :Template-driven Forms

T1.Import FormsModule into your app. Create a component called login-template with a simple login form (email and password fields) using ngModel for two-way binding.

Step 1: Generate the Component

Open the terminal in your project folder:

```
cd D:\Projects\my-first-app
```

Run:

```
ng generate component login-template
```

or

```
ng g c login-template
```

Step 2: Import FormsModule

Open:

```
src/app/login-template/login-template.ts
```

Replace it with:

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
```

```
@Component({
  selector: 'app-login-template',
  imports: [FormsModule],
  templateUrl: './login-template.html',
  styleUrls: ['./login-template.css']
})
export class LoginTemplate {
```

```
    email = "";
    password = "";
  }
```

T2. Add the name attribute to each input and wrap the form in ngForm. On submit, log the form's value to the console using (ngSubmit).

Open:

src/app/login-template/login-template.html

```
<h2>Login Form</h2>

<form #loginForm="ngForm" (ngSubmit)="onSubmit(loginForm)">

  <label>Email:</label>
  <br>

  <input
    type="email"
    name="email"
    [(ngModel)]="email">

  <br><br>

  <label>Password:</label>
  <br>

  <input
    type="password"
    name="password"
    [(ngModel)]="password">

  <br><br>

  <button type="submit">
    Login
  </button>

</form>
```

Step 3: Add Submit Method

Open:

login-template.ts

Add:

```
import { NgForm } from '@angular/forms';
```

Then add this method inside the class:

```
onSubmit(form: NgForm) {  
  console.log(form.value);  
}
```

The class should now contain:

```
email = "";  
password = "";  
  
onSubmit(form: NgForm) {  
  console.log(form.value);  
}
```

T3.Apply built-in validators (required, email, minlength) to the form fields and disable the Submit button until the form is valid.

```
<h2>Login Form</h2>
```

```
<form #loginForm="ngForm" (ngSubmit)="onSubmit(loginForm)">
```

```
<label>Email:</label>
```

```
<br>
```

```
<input  
  type="email"  
  name="email"  
  [(ngModel)]="email"  
  required  
  email  
  #emailField="ngModel">
```

```
<br><br>
```

```
<label>Password:</label>
```

```
<br>
```

```
<input
```

```

        type="password"
        name="password"
        [(ngModel)]="password"
        required
        minlength="6"
        #passwordField="ngModel">

<br><br>

<button
  type="submit"
  [disabled]="loginForm.invalid">
  Login
</button>

</form>

```

Now the **Login** button stays disabled until:

- Email is entered.
- Email is valid.
- Password is entered.
- Password has at least 6 characters.

T4. Display an inline error message below each field when it is touched and invalid (e.g., "Email is required").

```

<h2>Login Form</h2>

<form #loginForm="ngForm" (ngSubmit)="onSubmit(loginForm)">

  <label>Email:</label>
  <br>

  <input
    type="email"
    name="email"
    [(ngModel)]="email"
    required
    email
    #emailField="ngModel">

  <div *ngIf="emailField.invalid && emailField.touched">

```

```

    <small *ngIf="emailField.errors?.['required']">
      Email is required.
    </small>

    <small *ngIf="emailField.errors?.['email']">
      Enter a valid email.
    </small>

  </div>

  <br>

  <label>Password:</label>
  <br>

  <input
    type="password"
    name="password"
    [(ngModel)]="password"
    required
    minlength="6"
    #passwordField="ngModel">

  <div *ngIf="passwordField.invalid && passwordField.touched">

    <small *ngIf="passwordField.errors?.['required']">
      Password is required.
    </small>

    <small *ngIf="passwordField.errors?.['minlength']">
      Password must be at least 6 characters.
    </small>

  </div>

  <br>

  <button
    type="submit"
    [disabled]="loginForm.invalid">
    Login
  </button>

</form>

```

Step 5. Display the Component

Open:

src/app/app.html

```
<app-login-template></app-login-template>
```

Step 6. Import the Component into the Root Component

Open:

src/app/app.ts

Import the component:

```
import { LoginTemplate } from './login-template/login-template';
```

Then update the `imports` array:

```
@Component({  
  selector: 'app-root',  
  imports: [LoginTemplate],  
  templateUrl: './app.html',  
  styleUrls: ['./app.css']  
})
```

Step 7. Run the Project

```
ng serve -o
```

Open:

http://localhost:4200

Final Output:

Login Form

Email:

Password:

Password is required.

Login Form

Email:

Password:

Password must be at least 6 characters.

Login Form

Email:

Email is required.

Password:

Practical - 2 :Reactive Forms

R1.Import ReactiveFormsModule. Create a component called registration-reactive and build the same form (name, email, password) using FormGroup and FormControl directly in the TypeScript file.

Step 1: Generate the Component

Open the terminal:

```
ng g c registration-reactive
```

Step 2: Open

src/app/registration-reactive/registration-reactive.ts

```
import { Component } from '@angular/core';
import { ReactiveFormsModule, FormGroup, FormControl, Validators } from '@angular/forms';
import { CommonModule } from '@angular/common';
```

```
@Component({
  selector: 'app-registration-reactive',
  imports: [ReactiveFormsModule, CommonModule],
  templateUrl: './registration-reactive.html',
  styleUrls: ['./registration-reactive.css']
})
```

```
export class RegistrationReactive {
```

```
  registrationForm = new FormGroup({
```

```
    name: new FormControl("", [
      Validators.required
    ]),
```

```
    email: new FormControl("", [
      Validators.required,
      Validators.email
    ]),
```

```
    password: new FormControl("", [
      Validators.required,
      Validators.minLength(6)
    ])
  });
```

```

onSubmit() {
  console.log(this.registrationForm.value);
}

```

R2. Refactor the form from R1 to use FormBuilder instead of manually creating each FormControl. Apply the same built-in validators (required, email, minlength).

```

import { Component } from '@angular/core';
import {
  ReactiveFormsModule,
  FormBuilder,
  FormGroup,
  Validators
} from '@angular/forms';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-registration-reactive',
  imports: [ReactiveFormsModule, CommonModule],
  templateUrl: './registration-reactive.html',
  styleUrls: ['./registration-reactive.css']
})
export class RegistrationReactive {

  registrationForm!: FormGroup;

  constructor(private fb: FormBuilder) {
    this.registrationForm = this.fb.group({
      name: ['', Validators.required],
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required, Validators.minLength(6)]]
    });
  }

  onSubmit() {
    console.log(this.registrationForm.value);
  }

}

```

R3. Bind the FormGroup to the template using [formGroup] and FormControlName, and display validation error messages conditionally based on control state (touched, invalid, errors).

registration-reactive.html

```
<h2>Registration Form</h2>
```

```
<form
  [formGroup]="registrationForm"
  (ngSubmit)="onSubmit()">
```

```
  <label>Name</label>
  <br>
```

```
  <input
    type="text"
    formControlName="name">
```

```
  <div *ngIf="registrationForm.get('name')?.touched &&
    registrationForm.get('name')?.invalid">
```

```
    <small *ngIf="registrationForm.get('name')?.errors?.['required']">
      Name is required.
    </small>
```

```
  </div>
```

```
  <br><br>
```

```
  <label>Email</label>
  <br>
```

```
  <input
    type="email"
    formControlName="email">
```

```
  <div *ngIf="registrationForm.get('email')?.touched &&
    registrationForm.get('email')?.invalid">
```

```
    <small *ngIf="registrationForm.get('email')?.errors?.['required']">
      Email is required.
    </small>
```

```
    <small *ngIf="registrationForm.get('email')?.errors?.['email']">
      Enter a valid email.
    </small>
```

```
  </div>
```

```
<br><br>
```

```

<label>Password</label>
<br>
<input
  type="password"
  FormControlName="password">

<div *ngIf="registrationForm.get('password')?.touched &&
  registrationForm.get('password')?.invalid">

  <small *ngIf="registrationForm.get('password')?.errors?.['required']">
    Password is required.
  </small>

  <small *ngIf="registrationForm.get('password')?.errors?.['minlength']">
    Password must be at least 6 characters.
  </small>

</div>

<br><br>

<button
  type="submit"
  [disabled]="registrationForm.invalid">
  Register
</button>
</form>

```

Step 4. Display the Component

Open

src/app/app.html

Replace everything with:

```
<app-registration-reactive></app-registration-reactive>
```

Step 5. Import the Component

Open

src/app/app.ts

Import:

```
import { RegistrationReactive } from './registration-reactive/registration-reactive';
```

Update the component:

```
@Component({
  selector: 'app-root',
  imports: [RegistrationReactive],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
```

Step 6. Run the Project

ng serve -o

Test the Form

Try these cases:

Case 1

Leave everything empty.

Click inside each field, then click outside.

You should see:



The screenshot shows a web browser window with the address bar displaying 'localhost:4200'. The page title is 'Registration Form'. The form contains three input fields: 'Name', 'Email', and 'Password'. Each field has a red error message below it: 'Name is required.', 'Email is required.', and 'Password is required.' respectively. At the bottom of the form is a 'Register' button, which is currently disabled (grayed out).

Case 2

Email:

abc

You should see:

Registration Form

Name

Email

Enter a valid email.

Password

Register

Case 3

Password:

123

You should see:

Registration Form

Name

Email

Password

Password must be at least 6 characters.

Register

Case 4

Registration Form

Name

Email

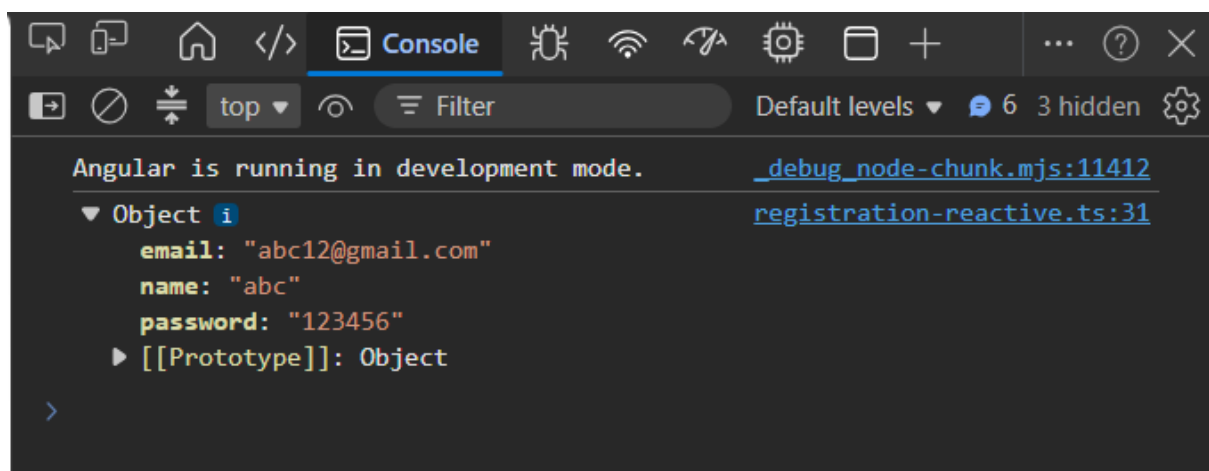
Password

The **Register** button becomes enabled.

Click **Register**.

Open F12 → Console.

You should see:



Practical - 3 :Custom & Cross-field Validation

V1.Write a custom validator function `noWhitespaceValidator` that rejects a value containing only spaces (or any spaces, your choice), and apply it to the username field of your Reactive Form.

Step 1: Create a Validator File

Inside:

`src/app`

Create a new file:

`validators.ts`

Step 2: Add the Validator

Paste this code into `validators.ts`:

```
import { AbstractControl, ValidationErrors } from '@angular/forms';

export function noWhitespaceValidator(
  control: AbstractControl
): ValidationErrors | null {

  const value = control.value;

  if (!value) {
    return null;
  }

  if (value.trim().length === 0) {
    return { whitespace: true };
  }

  return null;
}
```

Step 3: Import the Validator

Open:

registration-reactive.ts

Add:

```
import { noWhitespaceValidator } from '../validators';
```

Step 4: Apply It

Change the name control to:

```
name: [
  "",
  [
    Validators.required,
    noWhitespaceValidator
  ]
],
```

Step 5: Show the Error

In `registration-reactive.html`

Below the Name input, add:

```
<small
*ngIf="registrationForm.get('name')?.errors?.['whitespace']
&& registrationForm.get('name')?.touched">
```

Name cannot contain only spaces.

```
</small>
```

Test

Enter

" "

Only spaces.

You should see:

Registration Form

Name

Name cannot contain only spaces.

Email

Password

Register

V2.Add a Confirm Password field to your registration form and implement Cross-field Validation so the form is invalid if Password and Confirm Password don't match. Display an appropriate error message.

Step 1: Create Another Validator

Open

validators.ts

Add this below the first validator:

```
import { FormGroup, ValidationErrors } from '@angular/forms';
```

```
export function passwordMatchValidator(
```

```
  form: FormGroup
```

```
): ValidationErrors | null {
```

```
  const password = form.get('password')?.value;
```

```
  const confirmPassword =
```

```
    form.get('confirmPassword')?.value;
```

```
  if (password === confirmPassword) {
```

```
    return null;
```

```
  }
```

```
  return {
```

```
    passwordMismatch: true
```

```
  };
```

```
}
```

Step 2: Import It

Open

registration-reactive.ts

Add

```
import {  
  noWhitespaceValidator,  
  passwordMatchValidator  
} from '../validators';
```

Step 3: Add Confirm Password

Update your form:

```
this.registrationForm = this.fb.group({
```

```
  name: [  
    "",  
    [  
      Validators.required,  
      noWhitespaceValidator  
    ]  
  ],
```

```
  email: [  
    "",  
    [  
      Validators.required,  
      Validators.email  
    ]  
  ],
```

```
  password: [  
    "",  
    [  
      Validators.required,  
      Validators.minLength(6)  
    ]  
  ],
```

```

confirmPassword: [
  Validators.required
],
{
  validators: passwordMatchValidator
});

```

Step 4: Add Confirm Password Input

In

registration-reactive.html

Below Password add

```

<br><br>

<label>Confirm Password</label>

<br>

<input
type="password"
formControlName="confirmPassword">

<div
*ngIf="
registrationForm.get('confirmPassword')?.touched
&&
registrationForm.get('confirmPassword')?.invalid">

<small
*ngIf="
registrationForm.get('confirmPassword')
?.errors?.['required']">

Confirm Password is required.

</small>

</div>

```

Step 5: Show Password Mismatch Error

Below the Confirm Password section add

```
<div
*ngIf="
registrationForm.errors?.['passwordMismatch']
&&
registrationForm.get('confirmPassword')?.touched">

<small>

Passwords do not match.

</small>

</div>
```

Output:

Registration Form

Name

tom

Email

tom123@gmail.com

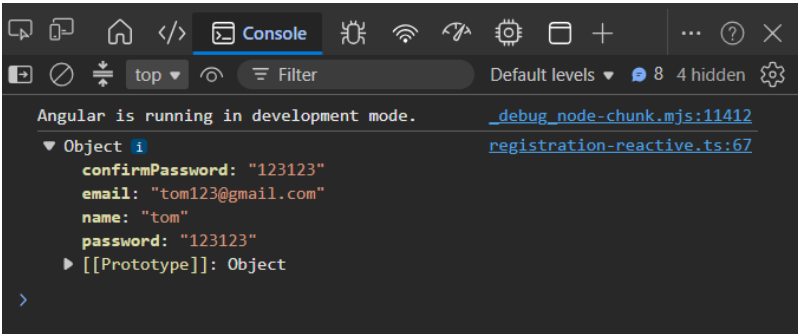
Password

.....

Confirm Password

.....

Register



Practical - 4 :Dynamic Forms

F1.Build a "Skills" section using FormArray where a user can dynamically add or remove skill input fields with Add/Remove buttons.

Step 1: Import FormArray

registration-reactive.ts

Update the imports:

```
import {  
  ReactiveFormsModule,  
  FormBuilder,  
  FormGroup,  
  FormArray,  
  Validators  
} from '@angular/forms';
```

Step 2: Add Skills FormArray

Inside the constructor, update the form:

```
this.registrationForm = this.fb.group({  
  
  name: [  
    "",  
    [  
      Validators.required,  
      noWhitespaceValidator  
    ]  
  ],  
  
  email: [  
    "",  
    [  
      Validators.required,  
      Validators.email  
    ]  
  ],  
  
  password: [  
    "",  
    [  

```

```

        Validators.required,
        Validators.minLength(6)
    ],
    confirmPassword: [
        "",
        Validators.required
    ],
    skills: this.fb.array([
        this.fb.control("")
    ])
},
{
    validators: passwordMatchValidator
});

```

Step 3: Create Getter

Inside the class add:

```

get skills(): FormArray {
    return this.registrationForm.get('skills') as FormArray;
}

```

Step 4: Add Skill Method

```

addSkill() {
    this.skills.push(this.fb.control(""));
}

```

Step 5: Remove Skill Method

```

removeSkill(index: number) {

    if (this.skills.length > 1) {
        this.skills.removeAt(index);
    }

}

```


F2. Submit the dynamic form and log the full array of entered skills to the console. Confirm the array updates correctly as fields are added/removed.

registration-reactive.html

Add this section **before the Register button**.

```
<hr>

<h3>Skills</h3>

<div formArrayName="skills">

  <div *ngFor="let skill of skills.controls; let i = index">

    <input
      type="text"
      [formControlName]="i"
      placeholder="Enter Skill">

    <button
      type="button"
      (click)="removeSkill(i)">
      Remove
    </button>

    <br><br>

  </div>

</div>

<button
  type="button"
  (click)="addSkill()">
  Add Skill
</button>

<br><br>
```

Step 6: Update Submit Method

```
onSubmit() {

  console.log("Complete Form");
```

```
console.log(this.registrationForm.value);

console.log("Skills");

console.log(this.skills.value);

}
```

Step 7: Run

ng serve -o

Output:

The screenshot displays a web browser at localhost:4200 showing a registration form. The form has fields for Name (filled with 'tom'), Email (filled with 'tom123@gmail.com'), Password (filled with '123123'), and Confirm Password (filled with '123123'). Below these is a 'Skills' section with three input fields: 'Java', 'Angular', and 'Python', each with a 'Remove' button. There is also an 'Add Skill' button and a 'Register' button at the bottom.

The browser's developer console is open, showing the following output:

```
Angular is running in development mode. _debug_node-chunk.mjs:11412
Complete Form registration-reactive.ts:87
  ▼ Object i registration-reactive.ts:89
    confirmPassword: "123123"
    email: "tom123@gmail.com"
    name: "tom"
    password: "123123"
    ► skills: (3) ['Java', 'Angular', 'Python']
    ► [[Prototype]]: Object

Skills registration-reactive.ts:91
  ▼ Array(3) i registration-reactive.ts:93
    0: "Java"
    1: "Angular"
    2: "Python"
    length: 3
    ► [[Prototype]]: Array(0)
```

Assignment - 5

Practical - 1 :Router Setup & Route Configuration

T1.Set up a standalone Angular app with routing using provideRouter in bootstrapApplication. Create three components: Home, About, and Contact.

Open the terminal in your project folder.

Run:

```
ng g c home
```

```
ng g c about
```

```
ng g c contact
```

```
ng g c page-not-found
```

You should now have:

```
src/app
```

```
|
```

```
├── home
```

```
├── about
```

```
├── contact
```

```
└── page-not-found
```

T2. Define routes for `'/'`, `'/about'`, and `'/contact'` mapping to the three components above. Add a `RouterOutlet` in your root component template.

Step 1: Create **app.routes.ts**

Inside **src/app**, create a file named:

`app.routes.ts`

Add:

```
import { Routes } from '@angular/router';

import { Home } from './home/home';
import { About } from './about/about';
import { Contact } from './contact/contact';
import { PageNotFound } from './page-not-found/page-not-found';

export const routes: Routes = [

  {
    path: '/',
    component: Home
  },

  {
    path: '/about',
    component: About
  },
```

```
{  
  path: 'contact',  
  component: Contact  
},  
  
{  
  path: '**',  
  component: PageNotFound  
}  
  
];
```

Step 2: Configure **app.config.ts**

Open

src/app/app.config.ts

Replace with:

```
import { ApplicationConfig } from '@angular/core';  
import { provideRouter } from '@angular/router';  
  
import { routes } from './app.routes';  
  
export const appConfig: ApplicationConfig = {  
  providers: [  
    provideRouter(routes)  
  ]  
};
```

Step 3: Check **main.ts**

It should contain:

```
import { bootstrapApplication } from '@angular/platform-browser';
```

```
import { appConfig } from './app/app.config';
```

```
import { App } from './app/app';
```

```
bootstrapApplication(App, appConfig)
```

```
.catch(err => console.error(err));
```

If your project was created with routing enabled, this is already correct.

T3.Build a navigation bar using routerLink for each of the three routes, and add routerLinkActive to highlight the currently active link.

Open

app.ts

Import RouterModule:

```
import { RouterModule } from '@angular/router';
```

Then

```
@Component({
```

```
  selector: 'app-root',
```

```
  imports: [
```

```
    RouterModule
```

```
],  
  templateUrl: './app.html',  
  styleUrls: ['./app.css']  
})
```

Open

app.html

Replace everything with:

```
<h1>Angular Routing Demo</h1>
```

```
<nav>
```

```
<a
```

```
  routerLink=""
```

```
  routerLinkActive="active"
```

```
  [routerLinkActiveOptions]="{ exact: true }">
```

```
    Home
```

```
</a>
```

```
|
```

```
<a
```

```
  routerLink="/about"
```

```
  routerLinkActive="active">
```

```
    About
```

```
</a>
```

|

```
<a
  routerLink="/contact"
  routerLinkActive="active">
  Contact
</a>
```

```
</nav>
```

```
<hr>
```

```
<router-outlet></router-outlet>
```

T4. Add a wildcard route ('**') that loads a `PageNotFoundComponent`, and confirm it only triggers for undefined URLs.

Open

app.css

Add:

```
nav a {
  text-decoration: none;
  margin-right: 15px;
  color: black;
}
```



```
.active{  
    color:red;  
    font-weight:bold;  
}
```

Step 5. Edit Home Component

Open

home.html

```
<h2>Home Page</h2>
```

```
<p>Welcome to the Home Page.</p>
```

Step 6. Edit About Component

Open

about.html

```
<h2>About Page</h2>
```

```
<p>This is the About Page.</p>
```

Step 7. Edit Contact Component

Open

contact.html

```
<h2>Contact Page</h2>
```

<p>This is the Contact Page.</p>

Step 8. Edit Page Not Found

Open

page-not-found.html

<h2>404</h2>

<p>Page Not Found</p>

Step 9. Run

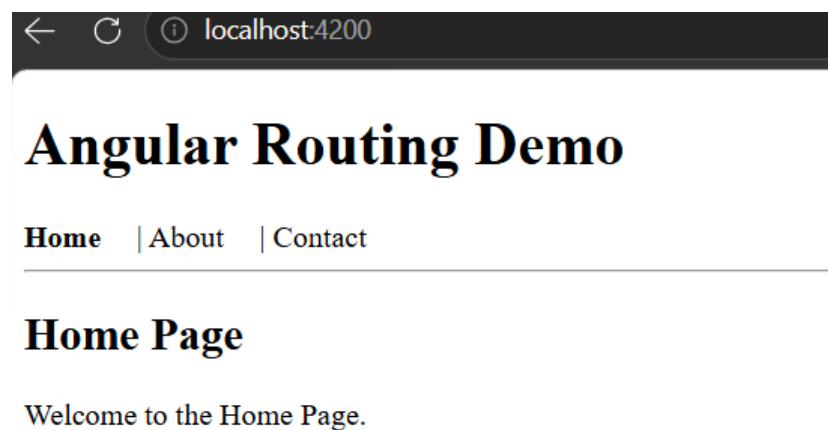
ng serve -o

Output:

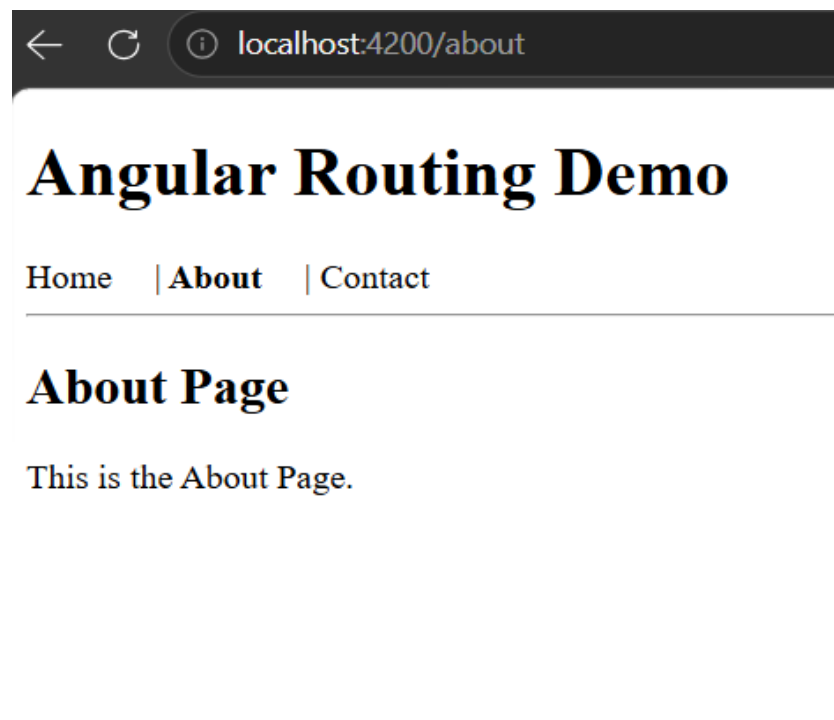
Visit

http://localhost:4200

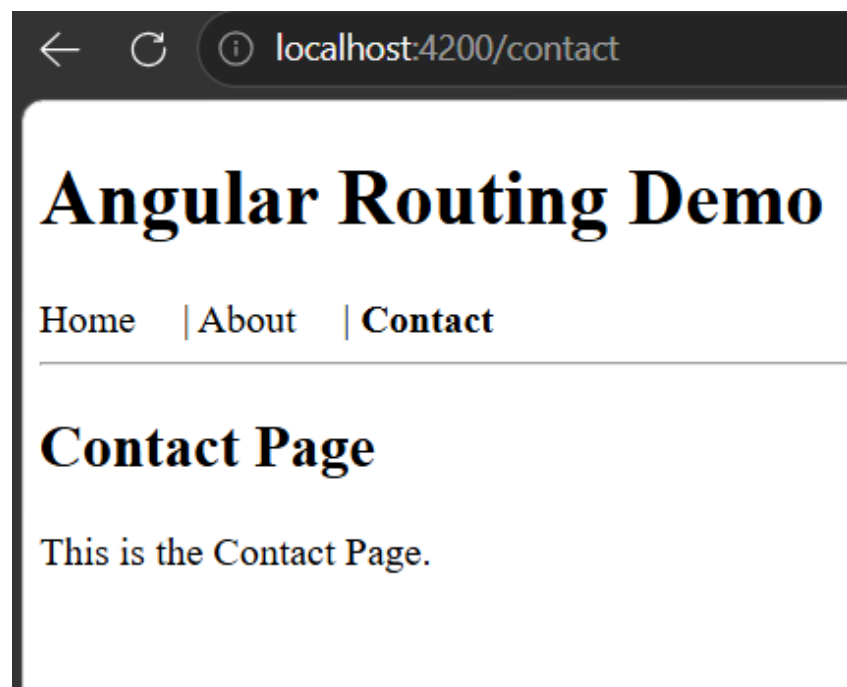
Home



About



Contact



Wildcard Route



This confirms the wildcard route (******) only handles undefined URLs.

Practical - 2 : Route Parameters & Query Parameters

P1. Create a UserComponent and configure a route 'users/:id'. Use ActivatedRoute and paramMap to read the id and display it in the template.

Step 1: Generate the Component

Open the terminal and run:

```
ng g c user
```

Step 2: Add the Route

Open:

```
src/app/app.routes.ts
```

Add this route **before** the wildcard (******) route:

```
{  
  path: 'users/:id',  
  component: User  
},
```

Import the component at the top:

```
import { User } from './user/user';
```

Your routes should now look like:

```
export const routes: Routes = [  
  { path: '', component: Home },  
  { path: 'about', component: About },  
  { path: 'contact', component: Contact },  
  { path: 'users/:id', component: User },  
  { path: '**', component: PageNotFound }  
]
```

```
];
```

Step 3: Update **user.ts**

Replace it with:

```
import { Component } from '@angular/core';
```

```
import { ActivatedRoute } from '@angular/router';
```

```
@Component({
```

```
  selector: 'app-user',
```

```
  imports: [],
```

```
  templateUrl: './user.html',
```

```
  styleUrls: ['./user.css']
```

```
})
```

```
export class User {
```

```
  userId: string | null = '';
```

```
  constructor(private route: ActivatedRoute) {
```

```
    this.route.paramMap.subscribe(params => {
```

```
      this.userId = params.get('id');
```

```
    });
```

```
  }}
```

Step 4: Update **user.html**

```
<h2>User Details</h2>
```

```
<p>User ID: {{ userId }}</p>
```

P2.Create a list of at least 5 users with routerLink directives that navigate to /users/:id for each one, and confirm the correct id displays when clicked.

We'll use the **Home** component.

Step 1: Open

home.ts

Replace with:

```
import { Component } from '@angular/core';
import { RouterModule } from '@angular/router';
import { CommonModule } from '@angular/common';
```

```
@Component({
  selector: 'app-home',
  imports: [RouterModule, CommonModule],
  templateUrl: './home.html',
  styleUrls: ['./home.css']
})
export class Home {
```

```
  users = [
    { id: 1, name: 'Dhruval' },
    { id: 2, name: 'Rahul' },
    { id: 3, name: 'Priya' },
    { id: 4, name: 'Amit' },
    { id: 5, name: 'Neha' }
  ];
}
```

Step 2: Open

home.html

```
<h2>User List</h2>

<ul>

  <li *ngFor="let user of users">

    <a [routerLink]="['/users', user.id]">

      {{ user.name }}

    </a>

  </li>

</ul>
```

Step 3: Run

ng serve -o

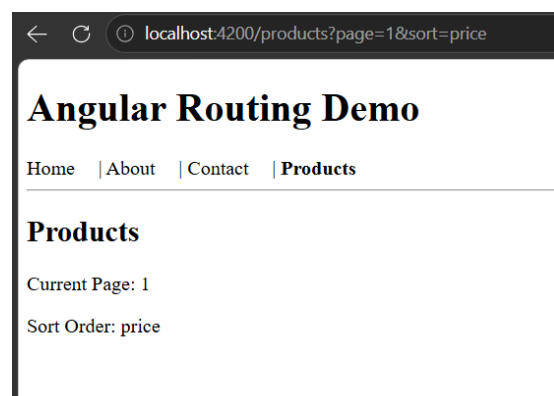
Output:

Angular Routing Demo

[Home](#) | [About](#) | [Contact](#)

User List

- [Dhruval](#)
- [Rahul](#)
- [Priya](#)
- [Amit](#)
- [Neha](#)



P3.Build a ProductsComponent with pagination controls. Use query parameters (e.g., ?page=2&sort=price) and queryParams to read and display the current page and sort order.

Step 1: Generate Component

ng g c products

Step 2: Add Route

In `app.routes.ts`

```
import { Products } from './products/products';
```

Add

```
{  
  path: 'products',  
  component: Products  
},
```

before the wildcard route.

Step 3: Update `products.ts`

```
import { Component } from '@angular/core';
```

```
import { ActivatedRoute } from '@angular/router';
```

```
@Component({  
  selector: 'app-products',  
  imports: [],  
  templateUrl: './products.html',  
  styleUrls: ['./products.css']  
})
```

```

export class Products {

  page = "";

  sort = "";

  constructor(private route: ActivatedRoute) {

    this.route.queryParamMap.subscribe(params => {

      this.page = params.get('page') ?? '1';

      this.sort = params.get('sort') ?? 'default';

    });

  }

}

```

Step 4: Update **products.html**

```

<h2>Products</h2>

<p>Current Page: {{ page }}</p>

<p>Sort Order: {{ sort }}</p>

```

Step 5: Add Link to Navigation

Open:

app.html

Add:

|

<a

routerLink="/products"

```
[queryParams]="{page:1,sort:'price'}"
```

```
routerLinkActive="active">
```

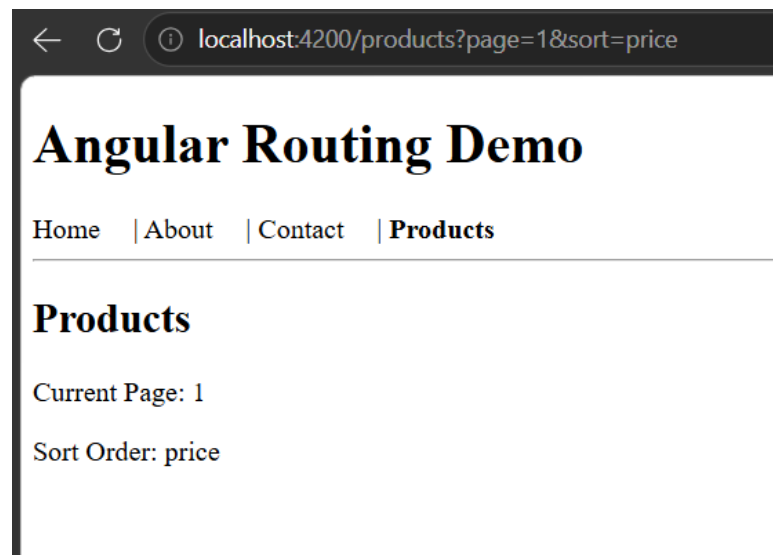
```
Products
```

```
</a>
```

Your navigation becomes:

Home | About | Contact | Products

Output:



Practical - 3 : Child Routes

C1.Build a simple Dashboard layout with a DashboardComponent containing its own RouterOutlet, and configure child routes for Users, Orders, and Settings.

C2.Add navigation links inside the dashboard for each child route and confirm the parent layout (header/sidebar) stays visible while only the child outlet content changes.

Step 1. Create Components

Open the terminal.

Run:

```
ng g c dashboard
```

```
ng g c dashboard/users
```

```
ng g c dashboard/orders
```

```
ng g c dashboard/settings
```

Your project should now contain:

```
src/app
├──
├── dashboard
│   ├── dashboard.ts
│   ├── dashboard.html
│   ├──
│   ├── users
│   ├── orders
│   └── settings
```

Step 2. Configure Child Routes

Open:

```
src/app/app.routes.ts
```

Import:

```
import { Dashboard } from './dashboard/dashboard';
```

```
import { Users } from './dashboard/users/users';
import { Orders } from './dashboard/orders/orders';
import { Settings } from './dashboard/settings/settings';
```

Add this route **before** the wildcard route:

```
{
  path: 'dashboard',
  component: Dashboard,
  children: [

    {
      path: 'users',
      component: Users
    },

    {
      path: 'orders',
      component: Orders
    },

    {
      path: 'settings',
      component: Settings
    },

    {
      path: '',
      redirectTo: 'users',
      pathMatch: 'full'
    }

  ]
},
```

Step 3. Update Dashboard Component

dashboard.ts

```
import { Component } from '@angular/core';
import { RouterModule } from '@angular/router';
```

```
@Component({
  selector: 'app-dashboard',
  imports: [RouterModule],
  templateUrl: './dashboard.html',
  styleUrls: ['./dashboard.css']
})
```

```
  })  
  export class Dashboard {  
  
  }
```

Step 4. Create Dashboard Layout

dashboard.html

Replace with:

```
<h2>Dashboard</h2>  
  
<hr>  
  
<nav>  
  
  <a  
    routerLink="users"  
    routerLinkActive="active">  
    Users  
  </a>  
  
  |  
  
  <a  
    routerLink="orders"  
    routerLinkActive="active">  
    Orders  
  </a>  
  
  |  
  
  <a  
    routerLink="settings"  
    routerLinkActive="active">  
    Settings  
  </a>  
  
</nav>  
  
<hr>  
  
<router-outlet></router-outlet>
```

Step 5. Import RouterModule

- `dashboard/users/users.ts`
- `dashboard/orders/orders.ts`
- `dashboard/settings/settings.ts`

users.ts

```
import { Component } from '@angular/core';
```

```
@Component({  
  selector: 'app-users',  
  imports: [],  
  templateUrl: './users.html',  
  styleUrls: ['./users.css']  
})  
export class Users {  
  
}
```

orders.ts

```
import { Component } from '@angular/core';
```

```
@Component({  
  selector: 'app-orders',  
  imports: [],  
  templateUrl: './orders.html',  
  styleUrls: ['./orders.css']  
})  
export class Orders {  
  
}
```

settings.ts

```
import { Component } from '@angular/core';
```

```
@Component({  
  selector: 'app-settings',  
  imports: [],  
  templateUrl: './settings.html',  
  styleUrls: ['./settings.css']  
})  
export class Settings {  
  
}
```

Step 6. Update HTML Files

users.html

```
<h3>Users Page</h3>
```

```
<p>This is the Users section.</p>
```

orders.html

```
<h3>Orders Page</h3>
```

```
<p>This is the Orders section.</p>
```

settings.html

```
<h3>Settings Page</h3>
```

```
<p>This is the Settings section.</p>
```

Step 7. Add Dashboard Link

Open:

app.html

Add:

```
|
```

```
<a  
routerLink="/dashboard"  
routerLinkActive="active">
```

Dashboard

```
</a>
```

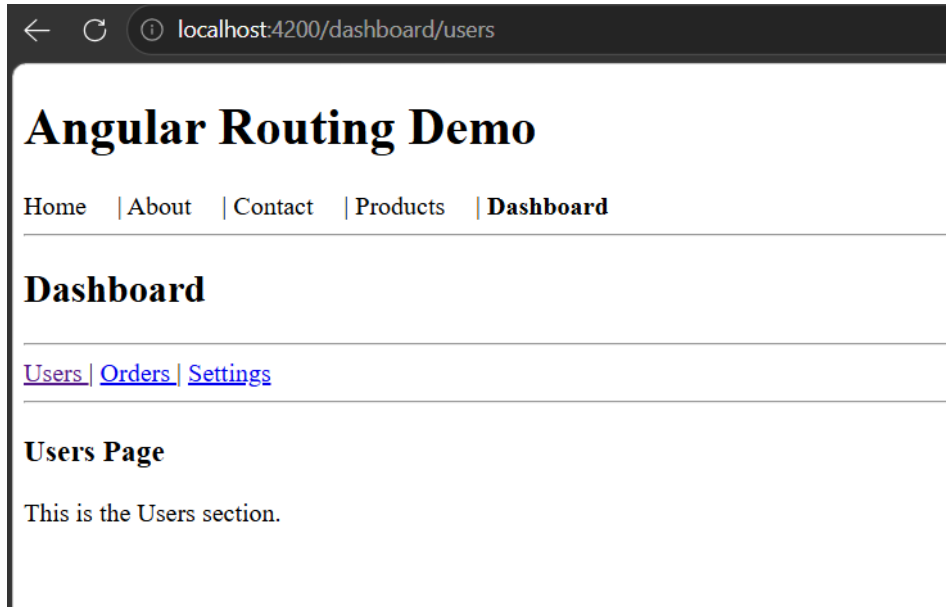
Now your navigation becomes:

Home | About | Contact | Products | Dashboard

Step 8. Run

ng serve -o

Output:



Practical - 4 : Lazy Loading

L1. Convert your About or Contact route to use `loadComponent` for lazy loading, and verify (using browser dev tools network tab) that its chunk loads only when the route is visited.

Step 1: Open `app.routes.ts`

Locate the About route.

It currently looks like:

```
{  
  path: 'about',  
  component: About  
},
```

Replace it with:

```
{  
  path: 'about',  
  loadComponent: () =>  
    import('./about/about').then(m => m.About)  
},
```

Step 2: Remove the Import

At the top of `app.routes.ts`, remove:

```
import { About } from './about/about';
```

The `About` component is now loaded only when needed.

Step 3: Run the Project

```
ng serve -o
```

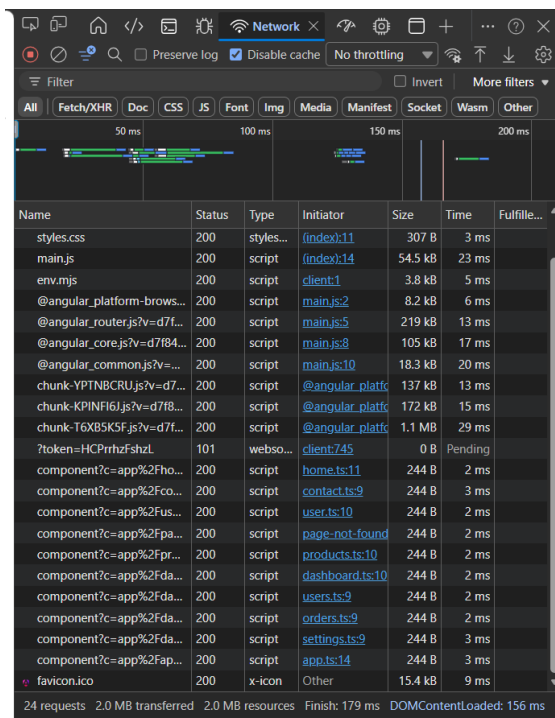
Step 4: Verify Lazy Loading

1. Open **Google Chrome**.
2. Press **F12**.
3. Open the **Network** tab.

4. Check **Disable cache** (optional but recommended).
5. Refresh the page.
6. Notice that the About component's JavaScript chunk is **not** loaded.
7. Click the **About** link.

A new JavaScript chunk for the About component will appear in the Network tab, confirming that it was loaded only when you navigated to that route.

Output: (Before clicking About)

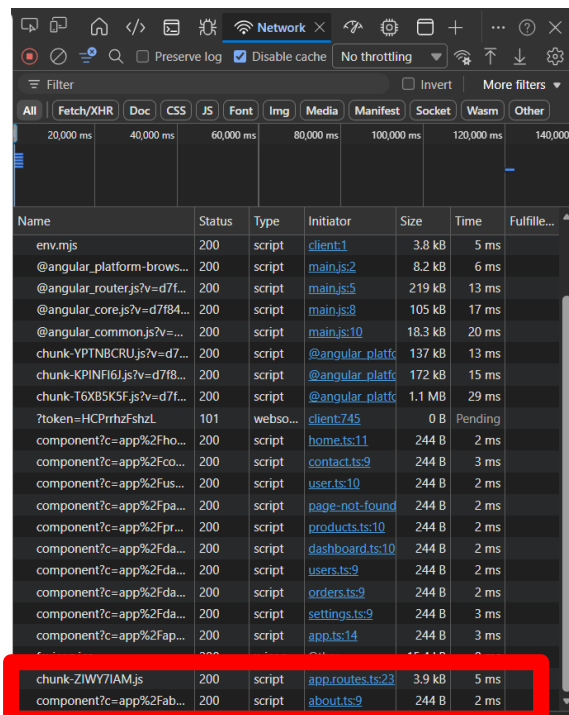


The screenshot shows the Chrome DevTools Network tab with the 'Disable cache' checkbox checked. The network log shows various resources loaded, including stylesheets, main.js, and several Angular chunks. The 'About' component's JavaScript chunk is not yet loaded.

Name	Status	Type	Initiator	Size	Time	Fulfil...
styles.css	200	styles...	/index:11	307 B	3 ms	
main.js	200	script	/index:14	54.5 kB	23 ms	
env.mjs	200	script	client:1	3.8 kB	5 ms	
@angular_platform-brows...	200	script	main.js:2	8.2 kB	6 ms	
@angular_router.js?v=d7f...	200	script	main.js:5	219 kB	13 ms	
@angular_core.js?v=d7f84...	200	script	main.js:8	105 kB	17 ms	
@angular_common.js?v=...	200	script	main.js:10	18.3 kB	20 ms	
chunk-YPTNBCRU.js?v=d7...	200	script	@angular_platf...	137 kB	13 ms	
chunk-KPINF6J.js?v=d7f8...	200	script	@angular_platf...	172 kB	15 ms	
chunk-T6XB5K5F.js?v=d7f...	200	script	@angular_platf...	1.1 MB	29 ms	
?token=HCPrrhZfshzL	101	webso...	client:745	0 B	Pending	
component?c=app%2Fcho...	200	script	home.ts:11	244 B	2 ms	
component?c=app%2Fco...	200	script	contact.ts:9	244 B	3 ms	
component?c=app%2Fus...	200	script	user.ts:10	244 B	2 ms	
component?c=app%2Fpa...	200	script	page-not-found	244 B	2 ms	
component?c=app%2Fpr...	200	script	products.ts:10	244 B	2 ms	
component?c=app%2Fda...	200	script	dashboard.ts:10	244 B	2 ms	
component?c=app%2Fda...	200	script	users.ts:9	244 B	2 ms	
component?c=app%2Fda...	200	script	orders.ts:9	244 B	2 ms	
component?c=app%2Fda...	200	script	settings.ts:9	244 B	3 ms	
component?c=app%2Fap...	200	script	app.ts:14	244 B	3 ms	
favicon.ico	200	x-icon	Other	15.4 kB	9 ms	

24 requests 2.0 MB transferred 2.0 MB resources Finish: 179 ms DOMContentLoaded: 156 ms

(After Clicking About)



The screenshot shows the Chrome DevTools Network tab after clicking the 'About' link. A new JavaScript chunk for the 'about' component has been loaded, highlighted with a red box.

Name	Status	Type	Initiator	Size	Time	Fulfil...
env.mjs	200	script	client:1	3.8 kB	5 ms	
@angular_platform-brows...	200	script	main.js:2	8.2 kB	6 ms	
@angular_router.js?v=d7f...	200	script	main.js:5	219 kB	13 ms	
@angular_core.js?v=d7f84...	200	script	main.js:8	105 kB	17 ms	
@angular_common.js?v=...	200	script	main.js:10	18.3 kB	20 ms	
chunk-YPTNBCRU.js?v=d7...	200	script	@angular_platf...	137 kB	13 ms	
chunk-KPINF6J.js?v=d7f8...	200	script	@angular_platf...	172 kB	15 ms	
chunk-T6XB5K5F.js?v=d7f...	200	script	@angular_platf...	1.1 MB	29 ms	
?token=HCPrrhZfshzL	101	webso...	client:745	0 B	Pending	
component?c=app%2Fcho...	200	script	home.ts:11	244 B	2 ms	
component?c=app%2Fco...	200	script	contact.ts:9	244 B	3 ms	
component?c=app%2Fus...	200	script	user.ts:10	244 B	2 ms	
component?c=app%2Fpa...	200	script	page-not-found	244 B	2 ms	
component?c=app%2Fpr...	200	script	products.ts:10	244 B	2 ms	
component?c=app%2Fda...	200	script	dashboard.ts:10	244 B	2 ms	
component?c=app%2Fda...	200	script	users.ts:9	244 B	2 ms	
component?c=app%2Fda...	200	script	orders.ts:9	244 B	2 ms	
component?c=app%2Fda...	200	script	settings.ts:9	244 B	3 ms	
component?c=app%2Fap...	200	script	app.ts:14	244 B	3 ms	
chunk-ZIYW7IAM.js	200	script	app.routes.ts:23	3.9 kB	5 ms	
component?c=app%2Fab...	200	script	about.ts:9	244 B	2 ms	

L2.Create a small 'Reports' feature with its own route file and load it using loadChildren at the feature level.

Step 1: Create the Reports Component

Run:

```
ng g c reports
```

Step 2: Create a Routes File

Create a new file:

```
src/app/reports.routes.ts
```

Add:

```
import { Routes } from '@angular/router';
import { Reports } from './reports/reports';

export const REPORT_ROUTES: Routes = [

  {
    path: '',
    component: Reports
  }

];
```

Step 3: Update **app.routes.ts**

Add this route before the wildcard route:

```
{
  path: 'reports',
  loadChildren: () =>
    import('./reports.routes').then(m => m.REPORT_ROUTES)
},
```

Step 4: Update **reports.html**

```
<h2>Reports</h2>
```

```
<p>Welcome to the Reports Feature.</p>
```

Step 5: Add Navigation Link

Open:

app.html

Add:

```
|
```

```
<a  
  routerLink="/reports"  
  routerLinkActive="active">
```

Reports

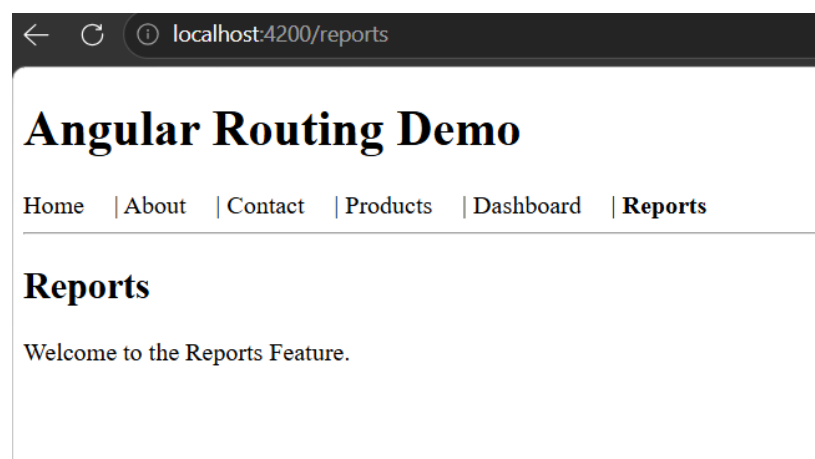
```
</a>
```

Now your navigation bar looks like:

Home | About | Contact | Products | Dashboard | Reports

Step 6: Run

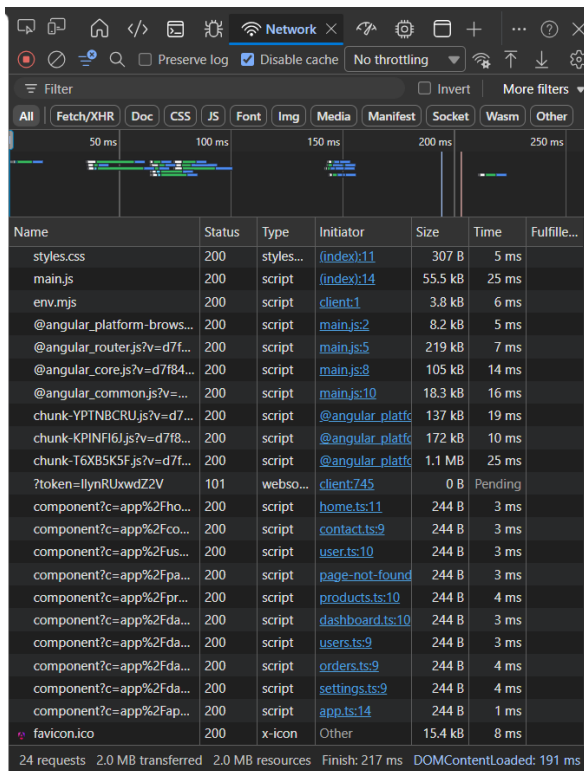
```
ng serve -o
```



Verify **loadChildren**

1. Open **Developer Tools (F12)**.
2. Go to the **Network** tab.
3. Refresh the application.
4. The Reports feature chunk is **not** downloaded initially.
5. Click **Reports**.
6. A new JavaScript chunk (for the Reports feature) appears in the Network tab, showing that the feature was loaded on demand.

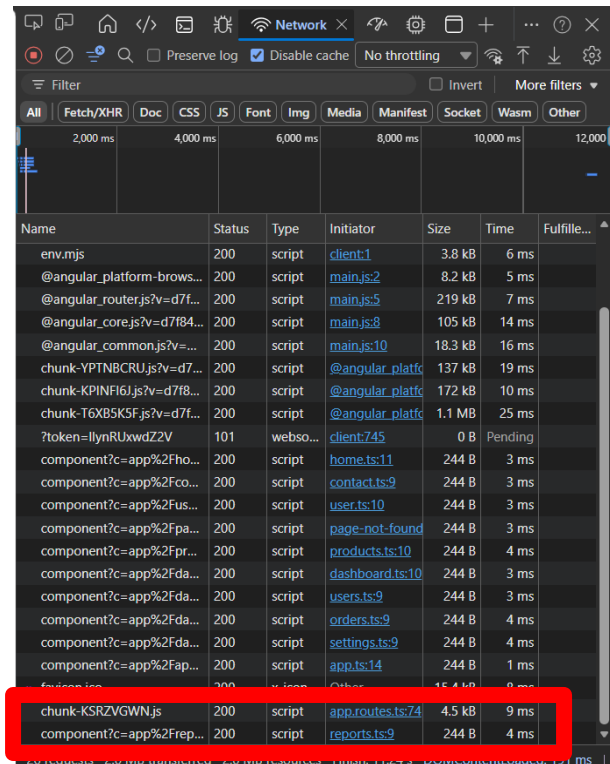
Output: (Before clicking Reports)



Name	Status	Type	Initiator	Size	Time	Fulfill...
styles.css	200	styles...	(index):11	307 B	5 ms	
main.js	200	script	(index):14	55.5 kB	25 ms	
env.mjs	200	script	client:1	3.8 kB	6 ms	
@angular_platform-brows...	200	script	main.js:2	8.2 kB	5 ms	
@angular_router.js?v=d7f...	200	script	main.js:5	219 kB	7 ms	
@angular_core.js?v=d7f84...	200	script	main.js:8	105 kB	14 ms	
@angular_common.js?v=...	200	script	main.js:10	18.3 kB	16 ms	
chunk-YPTNBCRU.js?v=d7...	200	script	@angular_platf...	137 kB	19 ms	
chunk-KPINF161.js?v=d7f8...	200	script	@angular_platf...	172 kB	10 ms	
chunk-T6XB5K5F.js?v=d7f...	200	script	@angular_platf...	1.1 MB	25 ms	
?token=llynRUxwdZ2V	101	webso...	client:745	0 B	Pending	
component?c=app%2Fho...	200	script	home.ts:11	244 B	3 ms	
component?c=app%2Fco...	200	script	contact.ts:9	244 B	3 ms	
component?c=app%2Fus...	200	script	user.ts:10	244 B	3 ms	
component?c=app%2Fpa...	200	script	page-not-found	244 B	3 ms	
component?c=app%2Fpr...	200	script	products.ts:10	244 B	4 ms	
component?c=app%2Fda...	200	script	dashboard.ts:10	244 B	3 ms	
component?c=app%2Fda...	200	script	users.ts:9	244 B	3 ms	
component?c=app%2Fda...	200	script	orders.ts:9	244 B	4 ms	
component?c=app%2Fda...	200	script	settings.ts:9	244 B	4 ms	
component?c=app%2Fap...	200	script	app.ts:14	244 B	1 ms	
favicon.ico	200	x-icon	Other	15.4 kB	8 ms	

24 requests 2.0 MB transferred 2.0 MB resources Finish: 217 ms DOMContentLoaded: 191 ms

(After Clicking Reports)



Name	Status	Type	Initiator	Size	Time	Fulfill...
env.mjs	200	script	client:1	3.8 kB	6 ms	
@angular_platform-brows...	200	script	main.js:2	8.2 kB	5 ms	
@angular_router.js?v=d7f...	200	script	main.js:5	219 kB	7 ms	
@angular_core.js?v=d7f84...	200	script	main.js:8	105 kB	14 ms	
@angular_common.js?v=...	200	script	main.js:10	18.3 kB	16 ms	
chunk-YPTNBCRU.js?v=d7...	200	script	@angular_platf...	137 kB	19 ms	
chunk-KPINF161.js?v=d7f8...	200	script	@angular_platf...	172 kB	10 ms	
chunk-T6XB5K5F.js?v=d7f...	200	script	@angular_platf...	1.1 MB	25 ms	
?token=llynRUxwdZ2V	101	webso...	client:745	0 B	Pending	
component?c=app%2Fho...	200	script	home.ts:11	244 B	3 ms	
component?c=app%2Fco...	200	script	contact.ts:9	244 B	3 ms	
component?c=app%2Fus...	200	script	user.ts:10	244 B	3 ms	
component?c=app%2Fpa...	200	script	page-not-found	244 B	3 ms	
component?c=app%2Fpr...	200	script	products.ts:10	244 B	4 ms	
component?c=app%2Fda...	200	script	dashboard.ts:10	244 B	3 ms	
component?c=app%2Fda...	200	script	users.ts:9	244 B	3 ms	
component?c=app%2Fda...	200	script	orders.ts:9	244 B	4 ms	
component?c=app%2Fda...	200	script	settings.ts:9	244 B	4 ms	
component?c=app%2Fap...	200	script	app.ts:14	244 B	1 ms	
chunk-KSRZVGWN.js	200	script	app.routes.ts:74	4.5 kB	9 ms	
component?c=app%2Frep...	200	script	reports.ts:9	244 B	4 ms	

24 requests 2.0 MB transferred 2.0 MB resources Finish: 142 ms DOMContentLoaded: 171 ms

Practical - 5 : Route Guards (AuthGuard)

G1. Create a simple AuthService with an isLoggedIn() method (you can hardcode it to return true/false for testing or toggle it with a button).

Step 1: Generate the Service

Open the terminal:

```
ng g s auth
```

This creates:

```
src/app/auth.ts
```

Step 2: Update **auth.ts**

```
import { Injectable } from '@angular/core';
```

```
@Injectable({
```

```
  providedIn: 'root'
```

```
})
```

```
export class Auth {
```

```
  loggedIn = false;
```

```
  isLoggedIn(): boolean {
```

```
    return this.loggedIn;
```

```
  }
```

```
  login() {
```

```
    this.loggedIn = true;
```

```
  }
```

```
  logout() {
```

```
    this.loggedIn = false;
```

```
  }}
```

G2.Implement an authGuard using CanActivateFn that calls AuthService.isLoggedIn(), and apply it to a protected 'profile' route using canActivate.

Generate it:

```
ng g c profile
```

Update **profile.html**

```
<h2>Profile Page</h2>
```

```
<p>Welcome! You are logged in.</p>
```

Generate:

```
ng g c login
```

Update **login.ts**

```
import { Component } from '@angular/core';
```

```
import { Auth } from '../auth';
```

```
@Component({
```

```
  selector: 'app-login',
```

```
  imports: [],
```

```
  templateUrl: './login.html',
```

```
  styleUrls: ['./login.css']
```

```
})
```

```
export class Login {
```

```
  constructor(private auth: Auth) {}
```



```
login() {  
    this.auth.login();  
}  
  
logout() {  
    this.auth.logout();  
}  
}
```

Update **login.html**

```
<h2>Login Page</h2>
```

```
<button (click)="login()">
```

Login

```
</button>
```

```
<button (click)="logout()">
```

Logout

```
</button>
```

```
<p>
```

Use the buttons above to change the login status.

```
</p>
```

Create the Auth Guard

Generate:

```
ng g guard auth
```

Choose:

CanActivate

auth-guard.ts

```
import { inject } from '@angular/core';

import { CanActivateFn, Router } from '@angular/router';

import { Auth } from './auth';

export const authGuard: CanActivateFn = () => {

  const auth = inject(Auth);

  const router = inject(Router);

  if (auth.isLoggedIn()) {

    return true;

  }

  router.navigate(['/login']);

  return false;

};
```

G3.Test both outcomes: confirm the ProfileComponent loads when isLoggedIn() returns true, and that navigation is blocked or redirected (e.g., to a login page) when it returns false.

Step 1: Update Routes

Open:

app.routes.ts

Import:

```
import { Profile } from './profile/profile';
```

```
import { Login } from './login/login';
```

```
import { authGuard } from './auth-guard';
```

Add these routes before the wildcard route:

```
{  
  path: 'login',  
  component: Login  
},
```

```
{  
  path: 'profile',  
  component: Profile,  
  canActivate: [authGuard]  
},
```

Step 2: Add Navigation

Open:

app.html

Add:

|

<a

routerLink="/login"

routerLinkActive="active">

Login

|

<a

routerLink="/profile"

routerLinkActive="active">

Profile

Navigation becomes:

Home | About | Contact | Products | Dashboard | Reports | Login | Profile

Step 3 : Run

ng serve -o

Test 1 (Not Logged In)

`loggedIn` is initially:

```
loggedIn = false;
```

Open:

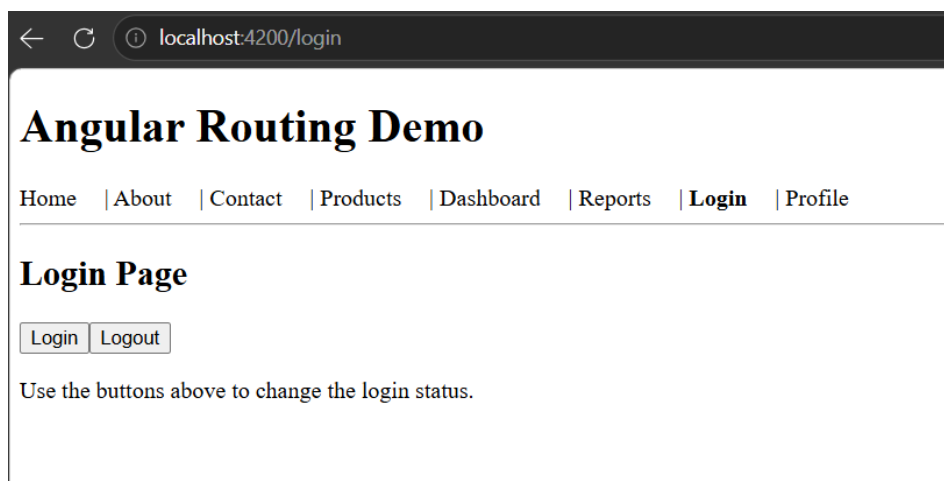
`http://localhost:4200/profile`

Result:

Profile page is blocked.

Automatically redirected to

`/login`



Test 2 (Logged In)

Visit:

`http://localhost:4200/login`

Click:

Login

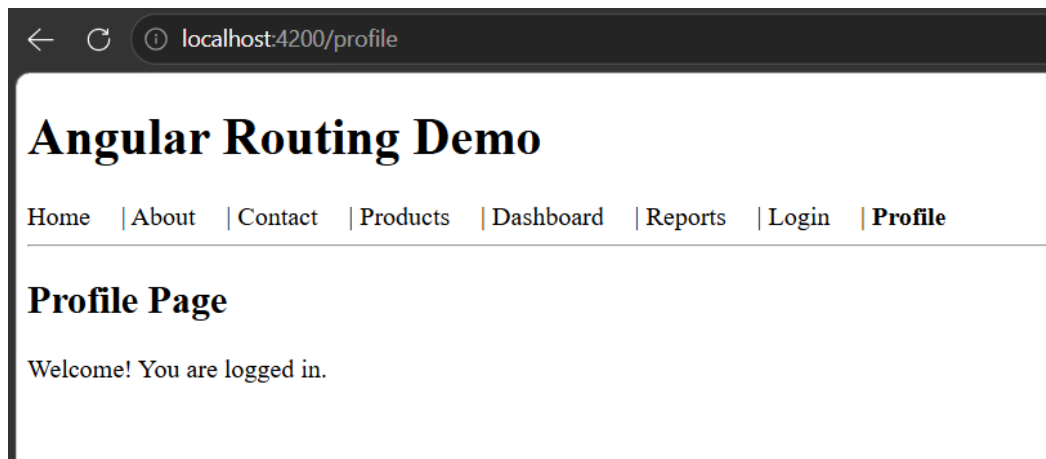
This sets:

```
loggedIn = true;
```

Now visit:

<http://localhost:4200/profile>

Output:



Test 3

Go back to: Login

Click

Logout

Now try /profile

Again you are redirected to

/login

